codigo.py
PY 6.4KB

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Vou analisar o arquivo passo a passo.

Análise do código

Passo 1: Identificação da estrutura

O arquivo é um script Python ofuscado que usa nomes de variáveis aleatórios. Vamos decodificar cada parte:

```python
aqgqzxkfjzbdnhz = __import__('base64')        # módulo base64
wogyjaaijwqbpxe = __import__('zlib')          # módulo zlib
idzextbcjbgkdih = 134                         # chave XOR = 134
qyrrhmmwrhaknyf = lambda ...                  # função de decodificação XOR
lzcdrtfxyqiplpd = 'eNq9W19z...'               # payload em base64+zlib
```

Passo 2: Entendendo a função lambda

```python
qyrrhmmwrhaknyf = lambda dfhulxliqohxamy, osatiehltgdbqxk: bytes([wtqiceobrebqsxl ^ idzextbcjbgkdih for wtqiceobrebqsxl in dfhulxliqohxamy])
```

Esta função aplica **XOR com a chave 134** a cada byte da entrada. É uma decodificação simples.

Passo 3: O fluxo de execução

1. `lzcdrtfxyqiplpd` contém uma string base64.
2. `base64.b64decode(...)` decodifica essa string.
3. `zlib.decompress(...)` descomprime o resultado.

4. `qyrrhmmwrhaknyf(...)` aplica XOR 134 no resultado descomprimido.
5. `exec(compile(...))` **executa o código resultante**.

Passo 4: O que o payload faz?

O padrão é clássico de **malware ofuscado**:

- **Ofuscação em camadas**: base64 → zlib → XOR → exec
- **Nomes de variáveis aleatórios** para dificultar análise
- `exec(compile(...))` executa código arbitrário em memória, sem gravar em disco
- **Importações dinâmicas** via `__import__`

Essas são técnicas típicas de **malware/loader** (também usadas em alguns scripts legítimos de proteção de código, mas com muito menos frequência nesse formato).

Passo 5: Tentativa de decodificar o payload

Não tenho como executar o código aqui com segurança, mas posso inferir: a string base64 começa com `eNq9...` — o prefixo `eNq` é a assinatura típica de dados comprimidos com **zlib** (header `78 9C` em base64 → `eNq...`). Isso confirma a estrutura.

Após decodificar e descomprimir, o XOR 134 revelaria o código Python real. Pelo tamanho da string (muito grande), o payload é substancial — consistente com um loader/backdoor/downloader.

Veredito

Este arquivo é altamente suspeito e deve ser tratado como malware.

Justificativas:

1. **Ofuscação em múltiplas camadas** (base64 + zlib + XOR) sem propósito legítimo aparente.
2. **Execução dinâmica via** `exec(compile(...))` de código decodificado em memória — técnica clássica de evasão.
3. **Nomes de variáveis aleatórios** para dificultar leitura humana e detecção por assinatura.
4. **Payload grande e oculto** que não pode ser inspecionado sem execução.

5. Ausência de qualquer contexto legítimo (biblioteca, aplicação, comentários, propósito).

Recomendação: Não execute este arquivo. Se precisar analisá-lo, faça em ambiente isolado (sandbox/VM sem rede), e submeta o hash a serviços como VirusTotal. O padrão observado é compatível com **loader/dropper de malware** (ex.: infostealer, backdoor ou ransomware).

Qual é a categoria principal deste malware?

Categoria principal: Loader / Dropper (carregador de segunda etapa)

Com base na estrutura do código, a categoria principal é **loader/dropper** — também chamado de **stager** ou **carregador de payload**.

Por que essa categoria?

O arquivo **não contém** o malware final em si de forma direta. Ele é um **invólucro** cuja única função é:

1. **Decodificar** um payload embutido (base64 → zlib → XOR).
2. **Executar** esse payload em memória via `exec(compile(...))`.

Ou seja, ele **carrega e executa** um segundo estágio. O comportamento real do malware (roubo de dados, backdoor, mineração, ransomware, etc.) está **oculto no payload decodificado**, que não é visível sem execução.

Categoria secundária (técnicas observadas)

- **Obfuscator/Packer** — ofuscação em camadas para evadir AV/EDR e análise estática.
- **Fileless execution** — o código roda em memória, sem gravar o payload em disco (dificulta forense e detecção).

O que não podemos afirmar sem decodificar o payload

Não é possível determinar a **família específica** (ex.: infostealer, RAT, ransomware, botnet) apenas pela camada externa, porque essa camada é apenas o "envelope". Para classificar a família, seria necessário:

- Decodificar o payload (base64 → zlib → XOR 134) em ambiente isolado;
- Analisar o código Python resultante (imports, URLs, comandos, persistência).

Resumo

Aspecto	Classificação
Categoria principal	Loader / Dropper (stager)
Técnicas	Ofuscação multi-camada, execução fileless via `exec`
Família específica	Indeterminada sem decodificar o 2º estágio
Risco	Alto — executa código arbitrário oculto

Recomendação: trate como malware; não execute fora de sandbox. Para identificar a família, decodifique o payload em ambiente controlado e submeta o hash (do arquivo e do payload decodificado) ao VirusTotal.